Next, we tell the order object to save itself (and its children, the line items) to the database. Along the way, the order object will perform validation (but we'll get to that in a minute).

If the save succeeds, we do two things. First, we ready ourselves for this customer's next order by deleting the cart from the session. Then we redisplay the catalog, using the `redirect_to` method to display a cheerful message. If, instead, the save fails, we redisplay the checkout form with the current cart.

In the `create` action, we assumed that the order object contains the `add_line_items_from_cart` method, so let's implement that method now:

rails7/depot_p/app/models/order.rb

```
  class Order < ApplicationRecord
    # ...
»   def add_line_items_from_cart(cart)
»     cart.line_items.each do |item|
»       item.cart_id = nil
»       line_items << item
»     end
»   end
  end
```

For each item that we transfer from the cart to the order, we need to do two things. First we set the `cart_id` to `nil` to prevent the item from going poof when we destroy the cart.

Then we add the item itself to the `line_items` collection for the order. Notice we didn't have to do anything special with the various foreign-key fields, such as setting the `order_id` column in the line item rows to reference the newly created order row. Rails does that knitting for us using the `has_many` and `belongs_to` declarations we added to the `Order` and `LineItem` models. Appending each new line item to the `line_items` collection hands the responsibility for key management over to Rails. We also need to modify the test to reflect the new redirect:

rails7/depot_p/test/controllers/orders_controller_test.rb